HUFE 数据结构第一讲

2026 年 3 月 14 日

1 本次课定位

本次课对应 5 次课安排中的第 1 次课，时长 3 小时，目标是完成**数据结构绪论、线性表、栈和队列**的基础梳理，并通过典型题建立做题入口。

1.1 本次课目标

- 理解数据、数据元素、数据结构、数据类型、抽象数据类型等基础概念。
- 区分逻辑结构与存储结构，建立“逻辑关系 + 存储实现 + 基本操作”的整体视角。
- 掌握线性表的定义，理解顺序表与链表的差异及常见考法。
- 掌握栈和队列的结构特征、基本操作、顺序与链式实现。
- 能处理基础复杂度判断、结构选择、循环队列判空判满、栈队列应用题。

1.2 考试导向提醒

这一讲虽然属于入门内容，但在试卷中出现频率很高，常见出题方式有三类：

- **概念判断题**：考查逻辑结构、存储结构、ADT、复杂度等基本概念。
- **比较分析题**：比较顺序表与链表、栈与队列、顺序存储与链式存储的优缺点。
- **基础计算题**：如顺序表插删复杂度、循环队列长度、栈的合法出栈序列判断。

因此复习时不要只背定义，还要形成“**定义 - 特点 - 操作 - 应用 - 易错点**”的完整链条。

1.3 3 小时建议节奏

阶段	时间	主要内容
第 1 段	30 分钟	绪论：基本概念、逻辑结构与存储结构、算法与复杂度入门
第 2 段	60 分钟	线性表：定义、ADT、顺序表、链表、典型比较题
第 3 段	70 分钟	栈和队列：定义、基本操作、顺序/链式实现、循环队列
第 4 段	20 分钟	栈和队列应用：括号匹配、表达式、层次遍历、调度

© 2026 isomoes

2 绪论

2.1 核心概念

1. **数据**：对客观事物的符号表示，是信息的载体。
2. **数据元素**：数据的基本单位，一个数据元素通常可由若干数据项组成。
3. **数据结构**：相互之间存在特定关系的数据元素的集合。
4. **数据类型**：一组性质相同的值及其上的操作的总称。
5. **抽象数据类型（ADT）**：从“逻辑功能”角度描述数据对象和操作，而不关心具体实现。

2.2 数据结构的三个层次

学习一个数据结构时，建议固定从以下三个角度去看：

- **逻辑结构**：元素之间是什么关系。
- **存储结构**：这种关系在计算机中怎样存。
- **基本操作**：查找、插入、删除、访问如何实现，代价如何。

2.3 逻辑结构分类

- 集合结构：元素之间除“同属一个集合”外无其他关系。
- 线性结构：元素之间是一对一关系。
- 树形结构：元素之间是一对多关系。
- 图形结构：元素之间是多对多关系。

2.4 存储结构分类

- 顺序存储：地址连续，支持随机访问。
- 链式存储：地址可不连续，依靠指针联系。
- 索引存储：在数据之外附加索引信息。
- 散列存储：根据关键字直接计算存储位置。

© 2026 isomoes

2.5 算法与复杂度入门

算法是求解问题的一组有穷、确定、可行的步骤。算法通常关注两个指标:

- **时间复杂度**: 执行操作次数随问题规模 n 增长的变化趋势。
- **空间复杂度**: 算法运行额外占用的存储空间规模。

常见时间复杂度从优到劣可记为:

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3)$$

2.6 复杂度理解例子

1. 顺序表按位访问第 i 个元素，只需一次地址计算，因此常看作 $O(1)$。
2. 在长度为 n 的顺序表表头插入一个元素，通常需要移动原有 n 个元素，因此为 $O(n)$。
3. 在单链表表头插入一个新结点，只需修改少量指针，因此若头指针已知，可看作 $O(1)$。
4. 在单链表中查找第 i 个结点，需要从头结点开始逐个后移，通常为 $O(n)$。

2.7 绪论部分常见判断

1. 逻辑结构和存储结构不是一一对应关系；同一逻辑结构可对应不同存储结构。
2. 抽象数据类型强调“做什么”，具体数据结构强调“怎么做”。
3. 时间复杂度只关注数量级，不关注常数项和低阶项。

2.8 绪论练习

1. 什么是数据结构？请用“数据元素关系”的角度作答。
2. 数据的逻辑结构与存储结构有什么区别?
3. 为什么说抽象数据类型和具体实现不是同一个层面的问题?
4. 请判断：线性结构一定采用顺序存储。(　　)
5. 请写出下列复杂度从小到大的排列：$O(n)$、$O(1)$、$O(n^2)$、$O(\log n)$。

3 线性表

3.1 定义

线性表是具有相同数据类型的 n 个数据元素的有限序列。
其特点是:

- 除第一个元素外，每个元素有且仅有一个直接前驱。
- 除最后一个元素外，每个元素有且仅有一个直接后继。
- 当 $n = 0$ 时，称为空表。

3.2 线性表的 ADT 视角

从考题角度，线性表的 ADT 常见操作包括：

- InitList：初始化
- ListEmpty：判空
- ListLength：求长度
- GetElem：按位查找
- LocateElem：按值查找
- ListInsert：插入
- ListDelete：删除

这里要注意两组容易混淆的操作：

- **按位查找**：根据位置找元素，如找第 i 个元素。
- **按值查找**：根据元素值找其所在位置或结点。

在顺序表中，按位查找优势明显；在链表中，两者通常都需要遍历。

3.3 顺序表

3.3.1 定义与特点

顺序表使用一组**地址连续**的存储单元依次存储线性表元素，因此逻辑上相邻的元素在物理上也相邻。

优点：

- 支持随机访问，按位查找快。
- 存储密度高，不需要额外指针域。

缺点：

- 插入和删除通常需要移动大量元素。
- 容量预先确定，扩展不灵活。

3.3.2 基本复杂度

操作	最好情况	最坏情况
按位查找	$O(1)$	$O(1)$
按值查找	$O(1)$	$O(n)$
插入	$O(1)$	$O(n)$
删除	$O(1)$	$O(n)$

3.3.3 典型考点

1. 在第 i 个位置插入元素时，需要将第 i 个及其后的元素整体后移。
2. 在表头插入或删除，最坏情况下都需要移动 n 级别元素。
3. 顺序表适合查询多、修改少的场景。

3.3.4 顺序表操作过程示意

设原顺序表为:

$$(a_1, a_2, a_3, a_4)$$

若在第 3 个位置插入元素 x，插入后得到:

$$(a_1, a_2, x, a_3, a_4)$$

此时原来的 a_3, a_4 都要向后移动一个位置。可见，**顺序表插入删除的核心代价往往来自元素移动，而不是赋值本身**。

3.4 链表

3.4.1 定义与特点

链表用一组**地址可以不连续**的存储单元存放元素，通过指针反映逻辑关系。

优点:

- 插入、删除时不需要整体移动元素。
- 空间可动态申请，扩展方便。

缺点:

- 不支持随机访问，按位查找通常要从头遍历。
- 每个结点需要额外指针域，存储密度较低。

3.4.2 常见类型

- 单链表：每个结点只有一个后继指针。
- 双链表：每个结点有前驱和后继两个指针。
- 循环链表：尾结点指针不为空，而是重新指向表头或头结点。

3.4.3 顺序表与链表对比

比较项目	顺序表	链表
存储空间	连续	不连续
按位访问	快，$O(1)$	慢，$O(n)$
插入删除	常需移动元素	常只需修改指针
存储密度	高	较低
适用场景	查询多	插删多

3.4.4 单链表插入与删除思想

若已知结点 p，要在其后插入新结点 s：

$$s \to next = p \to next, \quad p \to next = s$$

若删除结点 p 的后继结点 q：

$$p \to next = q \to next$$

注意：单链表删除某结点通常必须先找到其前驱结点。

3.4.5 链表操作过程理解

设有单链表：

$$head \to a \to b \to c \to NULL$$

若要在结点 b 后插入结点 x，操作顺序不能写反：

1. 先让 x 指向 b 的原后继，即 $x \to next = b \to next$；
2. 再让 b 指向 x，即 $b \to next = x$。

若先改第二步，就会丢失原后继结点地址。这类“**指针修改顺序题**”很容易在选择题和填空题中出现。

© 2026 isomoes

3.4.6 线性表部分的选型思路

当题目问“应该选择哪种结构”时，可以按下面思路判断：

- 若题目强调**随机访问、按位置快速读取**，优先考虑顺序表。
- 若题目强调**频繁插入删除、长度变化较大**，优先考虑链表。
- 若题目强调**存储密度高**，顺序表通常更优。
- 若题目强调**不要求连续空间**，链表更灵活。

3.5 线性表练习

1. 线性表的两个基本逻辑特征是什么？
2. 顺序表为什么适合按位查找？
3. 单链表为什么不适合频繁按位访问？
4. 在顺序表第 1 个位置插入元素，为什么最坏时间复杂度为 $O(n)$？
5. 在单链表中，若已知某结点前驱，则插入一个新结点的时间复杂度通常是多少？
6. 请判断：链表的插入操作一定比顺序表快。（　　）

4 栈和队列

4.1 栈

4.1.1 定义

栈是限定仅在表尾进行插入和删除的线性表。允许操作的一端称为**栈顶**，另一端称为**栈底**。栈的核心特征是：后进先出（LIFO）。

4.1.2 基本操作

- InitStack
- StackEmpty
- Push
- Pop
- GetTop

© 2026 isomoes

4.1.3 顺序栈

顺序栈通常使用数组和栈顶指针 `top` 实现。

```
#define MaxSize 100
typedef struct {
    ElemType data[MaxSize];
    int top;
} SqStack;
```

常见约定:

- 初始化: `top = -1`
- 栈空: `top == -1`
- 栈满: `top == MaxSize - 1`
- 进栈: 先 `top++`, 再存值
- 出栈: 先取值, 再 `top--`

如果栈中当前有 3 个元素, 则栈顶指针通常指向第 3 个元素的位置, 而不是下一个空位置。这一点要和某些教材中的不同约定区分开来。

4.1.4 链栈

链栈本质上是操作受限的单链表, 通常在链表头部进行入栈和出栈。

```
typedef struct LinkNode {
    ElemType data;
    struct LinkNode *next;
} LinkNode, *LinkStack;
```

链栈的优点是扩容灵活, 不需要预先分配固定长度。

4.1.5 栈的常见应用

- 括号匹配
- 表达式求值
- 函数调用与递归
- 浏览器前进后退

4.1.6 为什么表达式求值常用栈

表达式求值之所以常用栈，是因为运算具有“后出现的高优先级运算可能先处理”的特点，和栈的后进先出特征非常契合。

例如中缀表达式：

$$3 + 4 \times 2$$

虽然 $+$ 先出现，但计算时必须先处理 4×2。因此在实现中，常用一个栈保存运算符，必要时也会再用一个栈保存操作数。

4.1.7 栈的典型判断方法

判断一个序列是否可能为合法出栈序列时，最稳妥的方法不是凭感觉，而是**模拟入栈和出栈过程**：

1. 按给定顺序依次入栈；
2. 每入一个元素，就检查栈顶是否等于目标出栈序列当前元素；
3. 若相等则持续出栈；
4. 最终若目标序列可全部匹配，则该序列合法。

4.2 栈练习

1. 栈为什么称为操作受限的线性表?
2. 栈的逻辑特征是什么？请用一句话概括。
3. 顺序栈中，`top = -1` 一般表示什么状态?
4. 入栈序列为 $1, 2, 3$，请写出两个合法的出栈序列。
5. 为什么括号匹配问题通常使用栈而不是队列?
6. 请判断：函数递归调用过程中不需要栈空间。（　　）

4.3 队列

4.3.1 定义

队列是只允许在一端插入、另一端删除的线性表。插入的一端称为**队尾**，删除的一端称为**队头**。

队列的核心特征是：先进先出（FIFO）。

4.3.2 基本操作

- InitQueue
- QueueEmpty
- EnQueue
- DeQueue
- GetHead

4.3.3 循环队列是重点

普通顺序队列会出现“假溢出”问题，因此考试中更常考**循环队列**。

```c
#define MaxSize 100
typedef struct {
    ElemType data[MaxSize];
    int front, rear;
} SqQueue;
```

采用“牺牲一个存储单元”方案时：

- 初始化：`front = rear = 0`
- 队空：`front == rear`
- 队满：`(rear + 1) % MaxSize == front`
- 入队：先存入 `data[rear]`，再更新 `rear`
- 出队：先取 `data[front]`，再更新 `front`

这里的 `front` 一般指向当前队头元素的位置，`rear` 一般指向下一个可插入位置。正因为采用了这种定义，才需要牺牲一个存储单元来区分“空”和“满”。

队列长度公式：

$$\text{length} = (rear - front + MaxSize)\%MaxSize$$

4.3.4 循环队列计算示例

若 `MaxSize = 6`，当前 `front = 4`，`rear = 1`，则说明队列发生了“回绕”。此时长度为：

$$(1 - 4 + 6)\%6 = 3$$

这说明循环队列的长度计算不能简单用 `rear - front`，必须使用统一公式。

4.3.5 链队

链队通常带头结点，`front` 指向队头，`rear` 指向队尾。

- 入队在队尾进行。
- 出队在队头进行。
- 若最后一个元素出队，需让 `rear = front`。

4.3.6 队列的常见应用

- 层次遍历
- 广度优先搜索
- 作业调度
- 打印任务管理
- 消息缓冲

4.3.7 为什么层次遍历要用队列

二叉树层次遍历要求“先访问上一层，再访问下一层；同层中先左后右”。这正好符合队列的先进先出特征：

1. 根结点先入队；
2. 先出队的结点先访问；
3. 再把它的左右孩子按顺序入队；
4. 这样就能保证结点按层次顺序被访问。

4.4 队列练习

1. 队列为什么也是操作受限的线性表?
2. 队列的逻辑特征是什么?
3. 普通顺序队列为什么会出现“假溢出”?
4. 在循环队列中，采用牺牲一个存储单元的方法时，队空和队满条件分别是什么?
5. 若 `MaxSize = 8`，`front = 6`，`rear = 2`，则循环队列长度是多少?
6. 请判断：广度优先搜索通常使用队列实现。（　　）

© 2026 isomoes

5 易混点总结

1. **线性表、栈、队列的关系**：栈和队列都是特殊的线性表。
2. **顺序表和数组的关系**：顺序表通常借助数组实现，但顺序表强调的是一种线性表存储方式。
3. **链表按位访问慢**：因为逻辑位置无法直接映射为物理地址。
4. **循环队列判满**：若采用牺牲一个单元策略，不能把 `front == rear` 同时看作“空”和“满”。
5. **栈与递归**：递归的本质通常依赖栈保存现场。

6 本讲知识框架

建议把本讲内容整理成下面这条主线:

主题	核心问题	典型结构	高频考点
绪论	什么是数据结构	逻辑结构/存储结构	概念辨析、复杂度
线性表	线性关系如何存储	顺序表、链表	对比、选型、插删代价
栈	为什么后进先出	顺序栈、链栈	出栈序列、括号匹配
队列	为什么先进先出	循环队列、链队	判空判满、长度计算

7 典型例题

7.1 例题 1：顺序表插入的复杂度

题目： 在长度为 n 的顺序表第 1 个位置插入一个新元素，其时间复杂度是多少?
分析： 原有元素都要后移，共涉及 n 级别移动。
答案： $O(n)$。

7.2 例题 2：链表与顺序表选择

题目： 若某应用系统需要频繁插入和删除元素，而较少按位随机访问，应优先使用顺序表还是链表?
分析： 插删频繁时，链表不需要大量移动元素，更合适。
答案： 链表。

7.3 例题 3：栈的出栈序列判断

题目： 入栈序列为 $1, 2, 3, 4$，下列序列中哪一个不可能是出栈序列?
备选:

- A. $4, 3, 2, 1$
- B. $3, 4, 2, 1$

- C. $2, 4, 3, 1$
- D. $3, 1, 4, 2$

分析： D 不可能。因为 3 出栈后，1 若要先于 4 出栈，则 4 必须尚未入栈；但 2 已在 1 前入栈，之后再让 4 在 2 后出栈会矛盾。

答案： D。

7.4 例题 4：循环队列长度

题目： 循环队列中 `MaxSize = 8`，当前 `front = 2`，`rear = 6`，则队列中元素个数为多少？

解：

$$(6 - 2 + 8)\%8 = 4$$

答案： 4 个。

7.5 例题 5：链表删除的关键条件

题目： 在单链表中删除结点 p 的后继结点，操作的关键前提是什么？

分析： 必须保证结点 p 存在且其后继结点非空，否则无法完成删除。

答案： 已知待删结点的前驱结点，且前驱结点存在后继。

7.6 例题 6：结构选择题

题目： 某系统需要频繁读取第 i 个元素，但很少做插入和删除，应选链表还是顺序表？

分析： 频繁按位访问时，顺序表可直接通过地址计算定位。

答案： 顺序表。

8 课堂练习

8.1 基础练习

1. 数据结构从通常意义上看包括哪三个方面？
2. 逻辑结构可分为哪四类？
3. 顺序表按位查找为什么是 $O(1)$？
4. 单链表删除某结点时，为什么通常需要先找到其前驱？
5. 栈和队列分别满足什么原则？

8.2 判断与选择

1. 同一种逻辑结构只能对应一种存储结构。（　　）
2. 链表的插入和删除一定都是 $O(1)$。（　　）
3. 栈属于操作受限的线性表。（　　）
4. 若循环队列采用牺牲一个存储单元的方法，则 `front == rear` 表示队满。（　　）
5. 队列适合处理具有“先到先服务”特征的问题。（　　）
6. 栈顶指针 `top` 总是指向下一个可用空位置。（　　）

8.3 计算与分析

1. 一个顺序表长度为 n，在表尾插入元素的最好时间复杂度和最坏时间复杂度分别是多少?
2. 设循环队列 `MaxSize = 10`，`front = 7`，`rear = 2`，求当前队列长度。
3. 入栈序列为 a, b, c, d, e，请写出一个合法的出栈序列，并说明理由。
4. 简述顺序表与链表各自更适合什么应用场景。
5. 为什么说栈和队列都是特殊线性表，但它们的应用场景明显不同?
6. 若单链表中仅知道某结点本身，而不知道其前驱，删除操作会遇到什么困难?

8.4 综合小题

题目：某教务系统需要处理两个功能:

- 功能 A：按学号快速读取当前第 i 个学生记录;
- 功能 B：模拟打印任务，先提交的任务先打印。

请分别为功能 A 和功能 B 选择更合适的数据结构，并说明理由。

9 课后小测参考答案

1. 数据的逻辑结构、存储结构、基本操作。
2. 集合结构、线性结构、树形结构、图形结构。
3. 因为元素地址连续，可由首地址和位序直接计算存储地址。
4. 因为单链表只有后继指针，没有直接指向前驱的指针。
5. 栈是后进先出，队列是先进先出。
6. 第 1 题错；第 2 题错；第 3 题对；第 4 题错；第 5 题对；第 6 题错。

© 2026 isomoes

7. 当 `MaxSize = 10`，长度为 $(2-7+10)\%10=5$。

8. 栈强调“后进入先处理”，适合递归、回退、括号匹配；队列强调“先进入先处理”，适合调度、缓冲、层次遍历。

9. 因为单链表没有前驱指针，无法直接修改前驱结点的 `next` 域。

综合小题参考：功能 A 选顺序表，因为按位读取快；功能 B 选队列，因为打印任务符合先进先出原则。

9.1 分节练习参考答案

绪论练习参考：

1. 数据结构是相互之间存在特定关系的数据元素的集合。

2. 逻辑结构强调元素之间的逻辑关系；存储结构强调这些关系在计算机中的具体表示。

3. 因为 ADT 关注“提供哪些操作”，而具体实现关注“这些操作如何落地”。

4. 错。线性结构也可以采用链式存储。

5. $O(1) < O(\log n) < O(n) < O(n^2)$。

线性表练习参考：

1. 除首元素外每个元素有唯一前驱，除尾元素外每个元素有唯一后继。

2. 因为顺序表地址连续，可由首地址和位序直接计算存储位置。

3. 因为链表结点地址不连续，不能直接按位置定位，只能顺序遍历。

4. 因为原有元素通常都要整体后移。

5. 若前驱已知，通常为 $O(1)$。

6. 错。若链表插入前还要先查找位置，则整体代价未必更低。

栈练习参考：

1. 因为插入和删除只能在同一端进行。

2. 后进先出。

3. 空栈。

4. 如 $3,2,1$；$2,3,1$。

5. 因为最近遇到的左括号要最先和后续右括号匹配，符合后进先出。

6. 错。递归调用通常依赖栈保存返回地址和现场信息。

队列练习参考：

1. 因为插入和删除分别限制在两端进行。
2. 先进先出。
3. 因为多次出队后，数组前部虽有空位，但 `rear` 可能已到数组末端。
4. 队空：`front == rear`；队满：`(rear + 1) % MaxSize == front`。
5. 长度为 $(2-6+8)\%8=4$。
6. 对。

10 本讲小结

本讲是数据结构部分的入口，核心不是记定义，而是建立一个稳定的分析框架：**先看逻辑关系，再看存储实现，最后看基本操作和复杂度**。在线性表部分，要重点比较顺序表和链表；在栈和队列部分，要重点掌握其受限操作特征以及循环队列的判空、判满和长度计算。

下一讲将进入数组和广义表、树和二叉树，难度会明显上升，尤其要提前适应“性质 + 遍历 + 应用题”的出题方式。

© 2026 isomoes